

immutables: Persistent, Monoid-Annotated Finger Trees for R

Shawn T. O’Neil
TISLab

Abstract

We present **immutables**, an R package providing persistent finger-tree-backed collections (catenable sequences, priority queues, ordered sequences, and interval indices) with user-extensible monoid annotations. The same predicate-based locate and split primitives power every built-in operation and remain accessible to users, enabling $O(\log n)$ queries over custom aggregates. We describe the API, provide micro-benchmarks against base R and existing packages, and demonstrate the extension points with a DNA prefix-search example.

Keywords: R, finger trees, persistent data structures, monoids.

1. Introduction

Purely functional collections in R are rare: the stock `list` and `vector` types are semantically immutable but the ergonomic and asymptotic penalties for treating them so are severe. **immutables** closes that gap by providing a small suite of persistent collection types backed by 2-3 finger trees (Hinze and Paterson 2006) with user-extensible monoid annotations. The package unifies sequence, queue, priority-queue, ordered-sequence, and interval-index operations under a single tree representation and a single pair of predicate-driven primitives.

2. The package

immutables exposes four collection types:

- `flexseq` — a general persistent sequence supporting push/pop at both ends, indexing, split, and concatenation.
- `priority_queue` — a min- and max-priority queue.
- `ordered_sequence` — a key-ordered sequence with successor, predecessor, and range queries.
- `interval_index` — an interval tree supporting point and range overlap queries.

Internally each is a finger tree with a specific set of monoid annotations attached; the operations above delegate to two primitives — `locate_by_predicate()` and `split_by_predicate()` — over those annotations. Users are free to attach their own monoids via `add_monoids()` and invoke the same primitives for arbitrary $O(\log n)$ queries.

3. Benchmarks

Figures in this section are loaded from benchmark results produced by the replication script `data-raw/generate_publication_results.R` shipped with the package. The absolute numbers are platform-dependent; the shapes of the curves are the pertinent evidence.

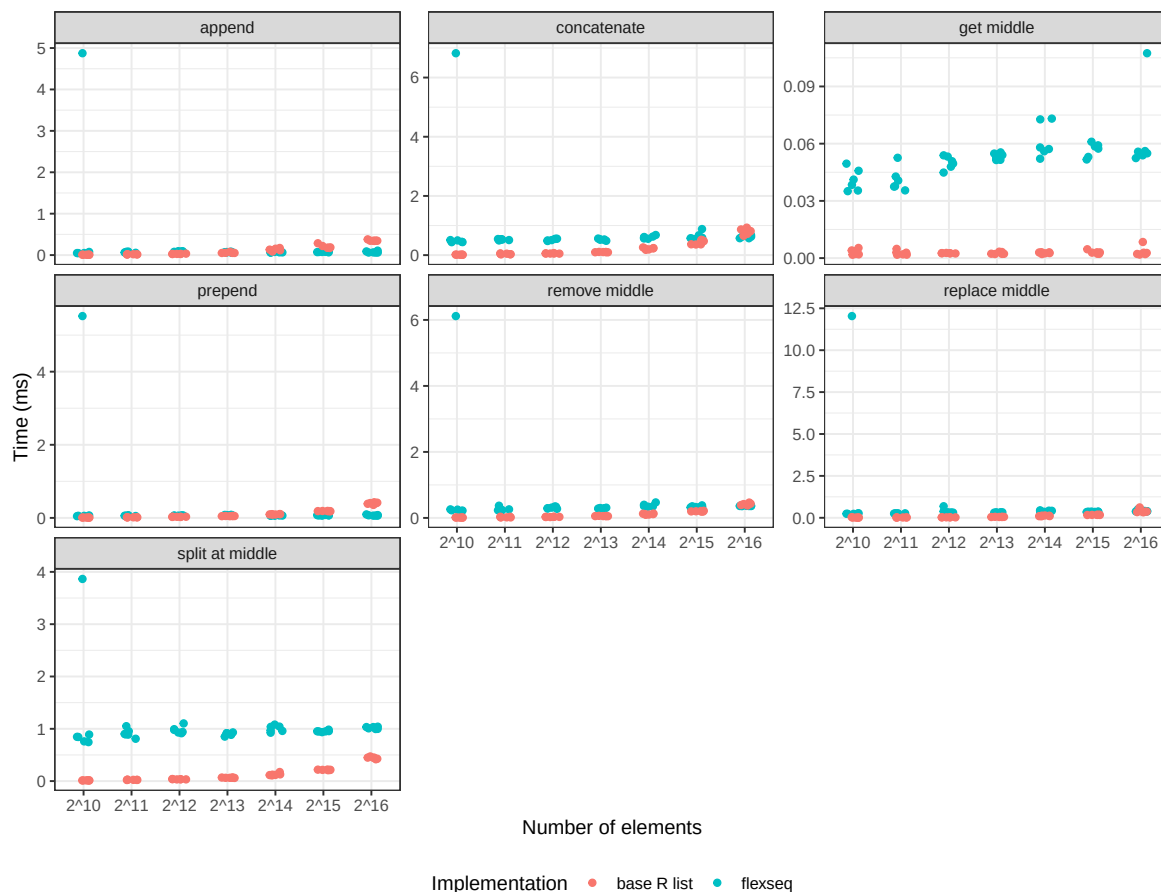


Figure 1: Sequence operations.

4. Extension: DNA prefix search

Custom monoids and predicate splits combine naturally for index-style queries. We build a persistent suffix-prefix index over a DNA string and use it to find all positions where a query pattern occurs as a prefix.

```
R> sequence <- "ACGCGCTCGCGCATAGTCGCGCCTG"
R> query <- "CGCGC"
```

Each position in the sequence is stored as a value keyed on a length-10 window starting at that position. Because `ordered_sequence` stores entries sorted by key, all matches to a given prefix lie in a contiguous run, so extracting them is two splits.

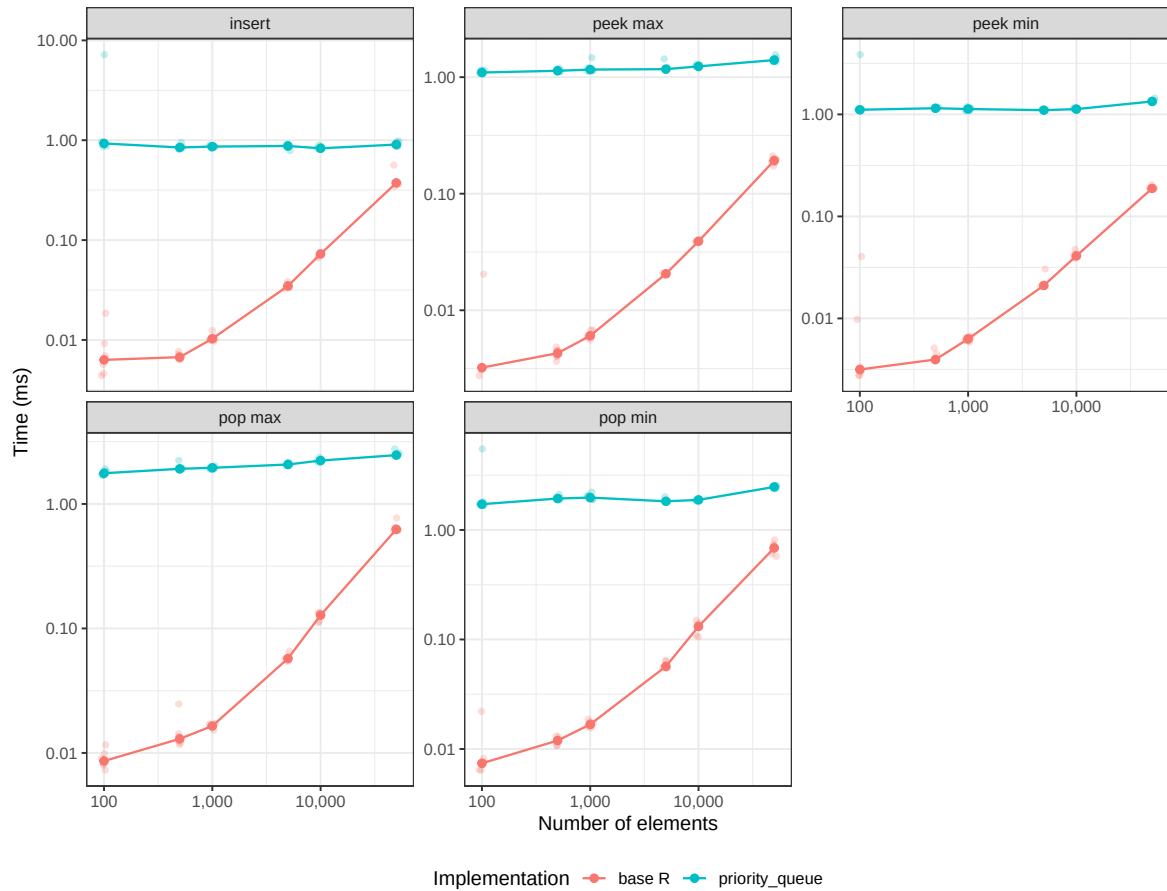


Figure 2: Priority queue operations.

We define a monoid tracking the maximum key (subsequence) seen so far and attach it to an empty `ordered_sequence`:

```
R> max_seq <- measure_monoid(max, i = -Inf, measure = function(entry) entry$key)
R> subseqs <- ordered_sequence()
R> subseqs <- add_monoids(subseqs, list(max_seq = max_seq))
```

Index every position by its length-10 subsequence:

```
R> for(i in seq_len(nchar(sequence))) {
+   subseqs <- insert(subseqs, i, key = substr(sequence, i, i + 10))
+ }
```

Split at the first entry whose key is at least the query:

```
R> first_split <- split_by_predicate(subseqs, function(m) query <= m, "max_seq")
```

If the right side is non-empty and its first key starts with the query, split again at the first key that is strictly greater *and* no longer starts with the query:

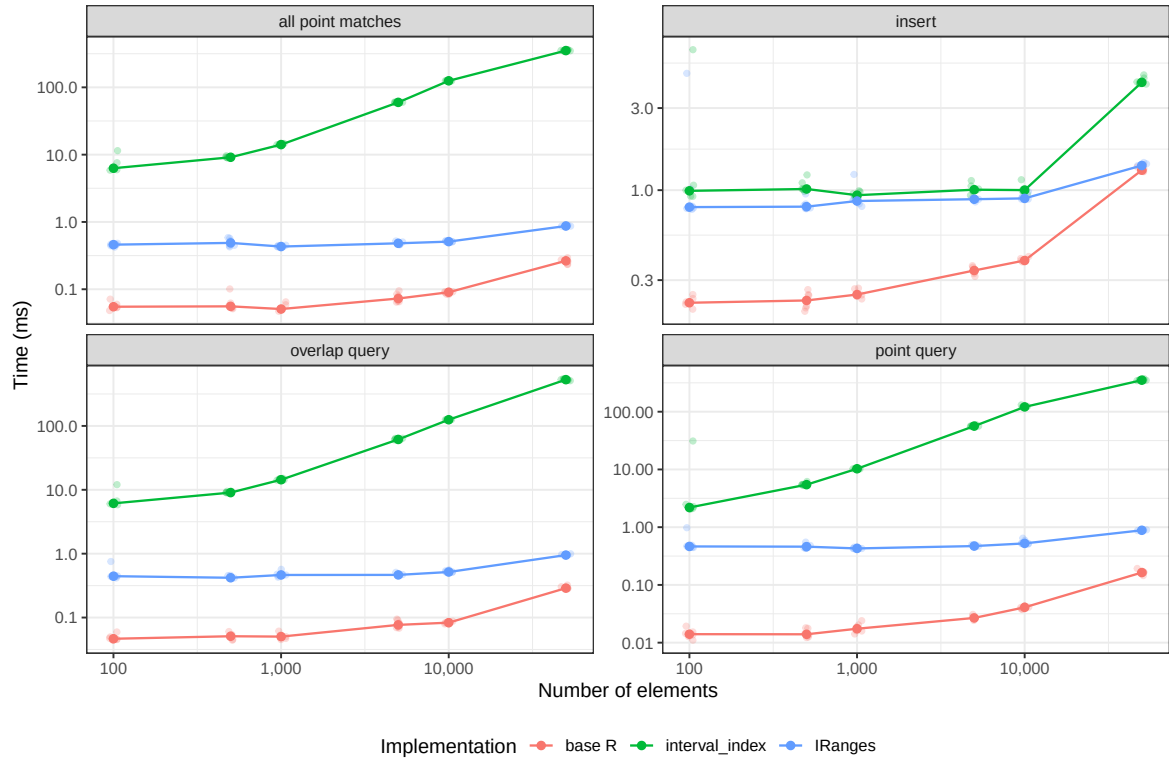


Figure 3: Interval index queries.

```
R> matches <- ordered_sequence()
R> if(length(first_split$right) > 0L) {
+   first_measure <- get_measures(first_split$right, "max_seq")[[1]]
+   if(startsWith(first_measure, query)) {
+     second_split <- split_by_predicate(
+       first_split$right,
+       function(m) query < m & !startsWith(m, query),
+       "max_seq"
+     )
+     matches <- second_split$left
+   }
+ }
R> positions <- fapply(matches, function(value, key) value) |> unlist()
R> positions
```

```
#> [1] 8 18 2
```

The two splits run in $O(\log n)$ each; recovering the matching positions from a k -element result run is $O(k)$. No auxiliary index structure is maintained beyond the cached monoid values attached to the tree nodes.

5. Conclusion

immutables provides the finger-tree vocabulary as a first-class extension surface rather than hiding it behind closed collection APIs. The cost of user-defined monoid attachments is small; the benefit is a uniform path to $O(\log n)$ operations over arbitrary subtree summaries.

Computational details

The results in this paper were obtained using R~4.3.0 with the **immutables** package at its current development version. Replication is via the script `data-raw/generate_publication_results.R` shipped with the package source.

Acknowledgments

References

Hinze R, Paterson R (2006). “Finger Trees: A Simple General-Purpose Data Structure.” *Journal of Functional Programming*, **16**(2), 197–217. doi:[10.1017/S0956796805005769](https://doi.org/10.1017/S0956796805005769).

Affiliation:

Shawn T. O’Neil

TISLab

Translational and Integrative Sciences Lab

E-mail: shawn@tislab.org